

Laporan Teknis: Analisis Sentimen Fitur SpayLater (ShopeePay)

Autolabeling dengan Google Gemini & Klasifikasi dengan SVM

(*Nama penulis / NIM*)

Notebook sumber: `spaylater_sentiment_analysis_gemini.ipynb`

April 12, 2026

Abstract

Laporan ini mendokumentasikan alur kerja pada notebook Jupyter untuk analisis sentimen teks media sosial terkait SpayLater/ShopeePay: prapemrosesan bahasa Indonesia (Sastrawi), pelabelan otomatis tiga kelas (*positif, netral, negatif*) memakai API Gemini, pelatihan model *Support Vector Machine* (SVM) dengan TF-IDF, evaluasi ketepatan pada data uji, serta visualisasi *wordcloud*. Setiap bagian kode dijelaskan dengan asumsi satu sel notebook setara dengan satu cuplikan layar kode.

Contents

1 Penjelasan per sel kode (cuplikan notebook)

Catatan: Nomor sel mengikuti indeks nol (`Cell 0`, `Cell 1`, ...) seperti urutan di `spaylater_sentiment_anal`. Hanya sel bertipe **kode** yang diberi cuplikan; sel Markdown berisi narasi di notebook dan tidak diduplikasi di sini sebagai gambar.

Sel 3 — Daftar dependensi (`requirements.txt`)

Tempat screenshot / cuplikan kode: Sel 3: menulis berkas `requirements.txt` berisi `google-generativeai`, `Sastrawi`, `pandas`, `scikit-learn`, `matplotlib`, `seaborn`, dan `wordcloud` agar lingkungan dapat direplikasi.

Berkas gambar disarankan: `figures/sel03_requirements`

Figure 1: Sel 3: menulis berkas `requirements.txt` berisi `google-generativeai`, `Sastrawi`, `pandas`, `scikit-learn`, `matplotlib`, `seaborn`, dan `wordcloud` agar lingkungan dapat direplikasi.

Tempat screenshot / cuplikan kode: Sel 4: perintah `pip install -r requirements.txt` untuk menginstal dependensi utama analisis.

Berkas gambar disarankan: `figures/sel04_ip_requirements`

Figure 2: Sel 4: perintah `pip install -r requirements.txt` untuk menginstal dependensi utama analisis.

Tempat screenshot / cuplikan kode: Sel 5: menginstal `mega.py` untuk mengunduh dataset dari tautan `Mega.nz` (opsional jika data sudah lokal).

Berkas gambar disarankan: `figures/sel05_ip_mega`

Figure 3: Sel 5: menginstal `mega.py` untuk mengunduh dataset dari tautan `Mega.nz` (opsional jika data sudah lokal).

*Tempat screenshot / cuplikan kode: Sel 6: memutakhirkan **tenacity** guna mengurangi konflik dependensi dengan **mega.py** atau paket lain di lingkungan yang sama.*

Berkas gambar disarankan: `figures/sel06iptenacity`

Figure 4: Sel 6: memutakhirkan **tenacity** guna mengurangi konflik dependensi dengan **mega.py** atau paket lain di lingkungan yang sama.

Tempat screenshot / cuplikan kode: Sel 7: login anonim Mega, mengunduh berkas Excel/CSV proyek dari URL yang ditentukan, dan mencetak path berkas hasil unduhan.

Berkas gambar disarankan: `figures/sel07megadownload`

Figure 5: Sel 7: login anonim Mega, mengunduh berkas Excel/CSV proyek dari URL yang ditentukan, dan mencetak path berkas hasil unduhan.

*Tempat screenshot / cuplikan kode: Sel 8: menetapkan variabel lingkungan **GEMINI_API_KEY** untuk autentikasi ke Google AI Studio / Gemini API. **Jangan** menyertakan kunci asli dalam laporan versi akhir; gunakan variabel lingkungan atau redaksi.*

Berkas gambar disarankan: `figures/sel08_apikey`

Figure 6: Sel 8: menetapkan variabel lingkungan **GEMINI_API_KEY** untuk autentikasi ke Google AI Studio / Gemini API. **Jangan** menyertakan kunci asli dalam laporan versi akhir; gunakan variabel lingkungan atau redaksi.

*Tempat screenshot / cuplikan kode: Sel 10: impor NumPy, pandas, scikit-learn, matplotlib, seaborn, WordCloud; konfigurasi path data (**CSV_PATH**), folder ekspor (**EXPORT_DIR**, **RAW_RESPONSE_DIR**), batas batch LLM (**MAX_TOKEN_DATA**, **MAX_LLM_BATCH_ITEMS**), model Gemini (**LLM_MODEL**), token keluaran, dan jeda antar batch (**LLM_BATCH_SLEEP_SEC**).*

Berkas gambar disarankan: `figures/sel10_config`

Figure 7: Sel 10: impor NumPy, pandas, scikit-learn, matplotlib, seaborn, WordCloud; konfigurasi path data (**CSV_PATH**), folder ekspor (**EXPORT_DIR**, **RAW_RESPONSE_DIR**), batas batch LLM (**MAX_TOKEN_DATA**, **MAX_LLM_BATCH_ITEMS**), model Gemini (**LLM_MODEL**), token keluaran, dan jeda antar batch (**LLM_BATCH_SLEEP_SEC**).

Tempat screenshot / cuplikan kode: Sel 11: mendefinisikan `count_tokens` memakai `tiktoken (cl100k_base)` jika tersedia; jika tidak, perkiraan kasar $\max(1, \lfloor |s|/4 \rfloor$) untuk membantu membagi batch LLM.

Berkas gambar disarankan: `figures/sel11_tiktoken`

Figure 8: Sel 11: mendefinisikan `count_tokens` memakai `tiktoken (cl100k_base)` jika tersedia; jika tidak, perkiraan kasar $\max(1, \lfloor |s|/4 \rfloor$) untuk membantu membagi batch LLM.

Tempat screenshot / cuplikan kode: Sel 13: membaca Excel dengan `pandas`, menampilkan cuplikan baris awal, dan memastikan kolom `full_text` ada.

Berkas gambar disarankan: `figures/sel13_read_excel`

Figure 9: Sel 13: membaca Excel dengan `pandas`, menampilkan cuplikan baris awal, dan memastikan kolom `full_text` ada.

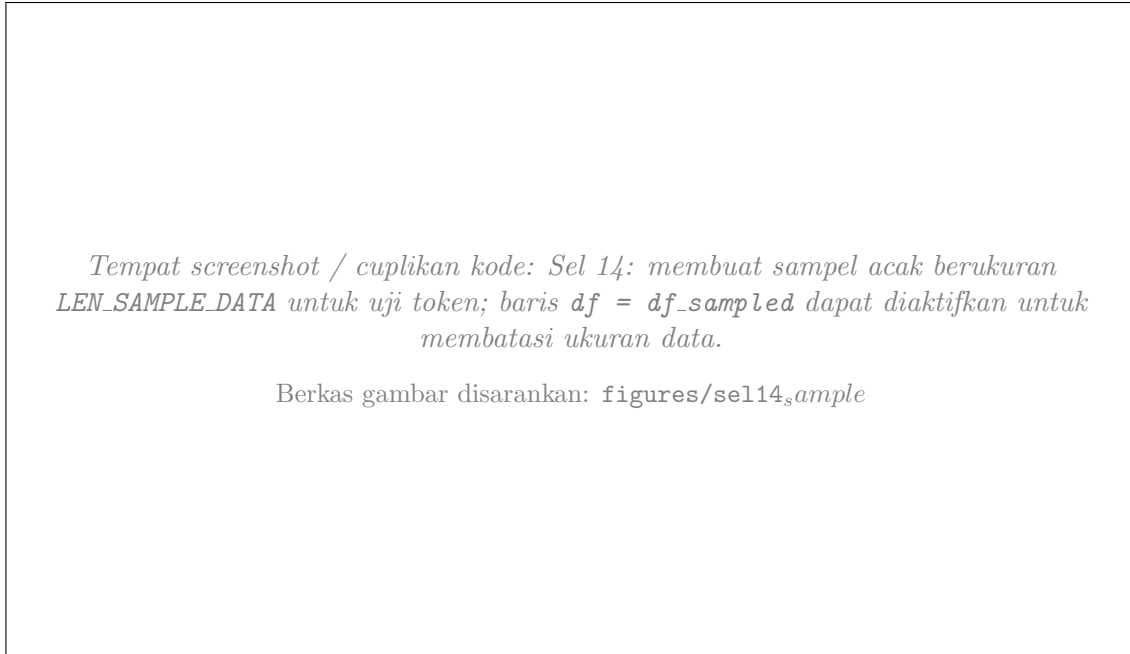


Figure 10: Sel 14: membuat sampel acak berukuran `LEN_SAMPLE_DATA` untuk uji token; baris `df = df_sampled` dapat diaktifkan untuk membatasi ukuran data.

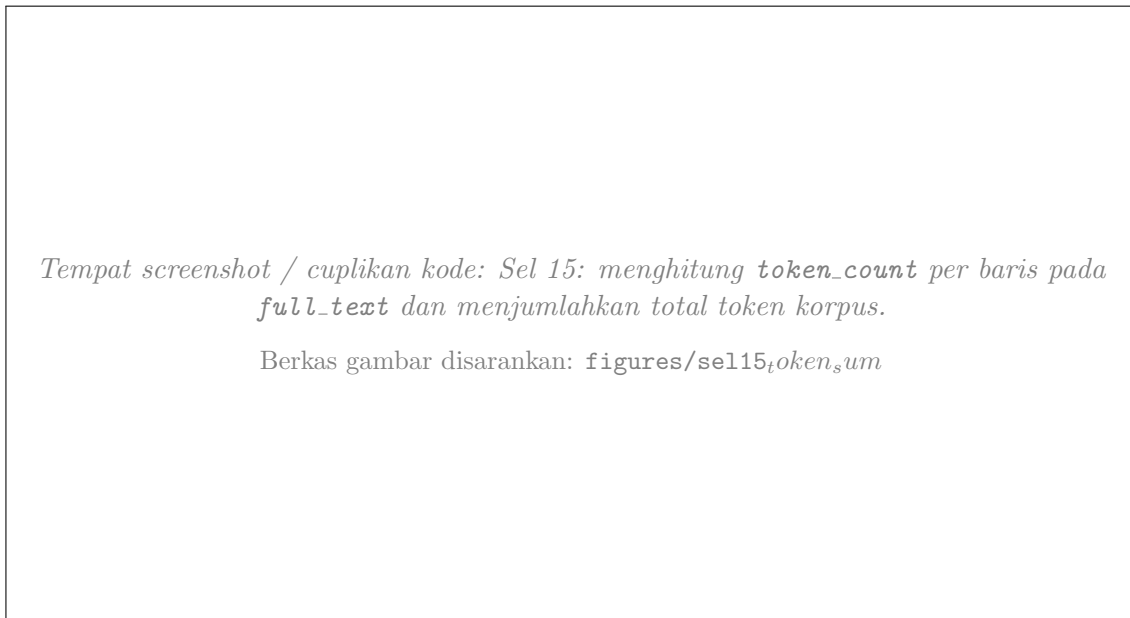


Figure 11: Sel 15: menghitung `token_count` per baris pada `full_text` dan menjumlahkan total token korpus.

Tempat screenshot / cuplikan kode: Sel 16: operasi aritmetika sederhana (mis. perbandingan total token terhadap sampel) untuk estimasi skala data.

Berkas gambar disarankan: `figures/sel16_ratio`

Figure 12: Sel 16: operasi aritmetika sederhana (mis. perbandingan total token terhadap sampel) untuk estimasi skala data.

Tempat screenshot / cuplikan kode: Sel 17: `len(df)` untuk mencetak jumlah observasi.

Berkas gambar disarankan: `figures/sel17_len`

Figure 13: Sel 17: `len(df)` untuk mencetak jumlah observasi.

Tempat screenshot / cuplikan kode: Sel 18: histogram distribusi token_count untuk melihat sebaran panjang teks.

Berkas gambar disarankan: `figures/sel18_hist`

Figure 14: Sel 18: histogram distribusi `token_count` untuk melihat sebaran panjang teks.

Sel 4 — Instalasi paket dari `requirements.txt`

Sel 5 — Paket unduhan Mega

Sel 6 — Penyesuaian `tenacity`

Sel 7 — Unduh dataset dari Mega

Sel 8 — Kunci API Gemini

Sel 10 — Impor pustaka dan konfigurasi global

Sel 11 — Estimasi token (`count_tokens`)

Sel 13 — Muat dataset

Sel 14 — Sampel acak (opsional)

Sel 15 — Total token pada seluruh baris

Sel 16 — Rasio estimasi token

Sel 17 — Jumlah baris

Sel 18 — Histogram panjang token

Sel 20 — Fungsi prapemrosesan teks

Sel 22 — Demo langkah prapemrosesan

Sel 24 — Terapkan prapemrosesan ke seluruh data

Sel 25 — Penanda / sel cadangan

Sel 27 — Autolabel dengan Gemini (inti)

Sel 29 — Nilai unik label

Sel 30 — Pelatihan SVM dan pencarian hyperparameter

Sel 32 — Matriks kebingungan dan laporan klasifikasi

Sel 34 — Wordcloud

Sel 36 — Ekspor artefak dan ZIP

Sel 37 — Unduh ZIP (Google Colab)

*Tempat screenshot / cuplikan kode: Sel 20: kamus singkatan (**SLANG_MAP**), regex **URL/email/mention/hashtag**, normalisasi Unicode, stemming dan stopword Sastrawi, serta **preprocess_series** untuk diterapkan ke kolom **DataFrame**.*

Berkas gambar disarankan: `figures/sel20_preprocess`

Figure 15: Sel 20: kamus singkatan (**SLANG_MAP**), regex **URL/email/mention/hashtag**, normalisasi Unicode, stemming dan stopword Sastrawi, serta **preprocess_series** untuk diterapkan ke kolom **DataFrame**.

*Tempat screenshot / cuplikan kode: Sel 22: fungsi **demo_steps** mencetak tiap tahap prapemrosesan untuk beberapa sampel pertama (**full_text**) guna verifikasi visual.*

Berkas gambar disarankan: `figures/sel22_demo`

Figure 16: Sel 22: fungsi **demo_steps** mencetak tiap tahap prapemrosesan untuk beberapa sampel pertama (**full_text**) guna verifikasi visual.

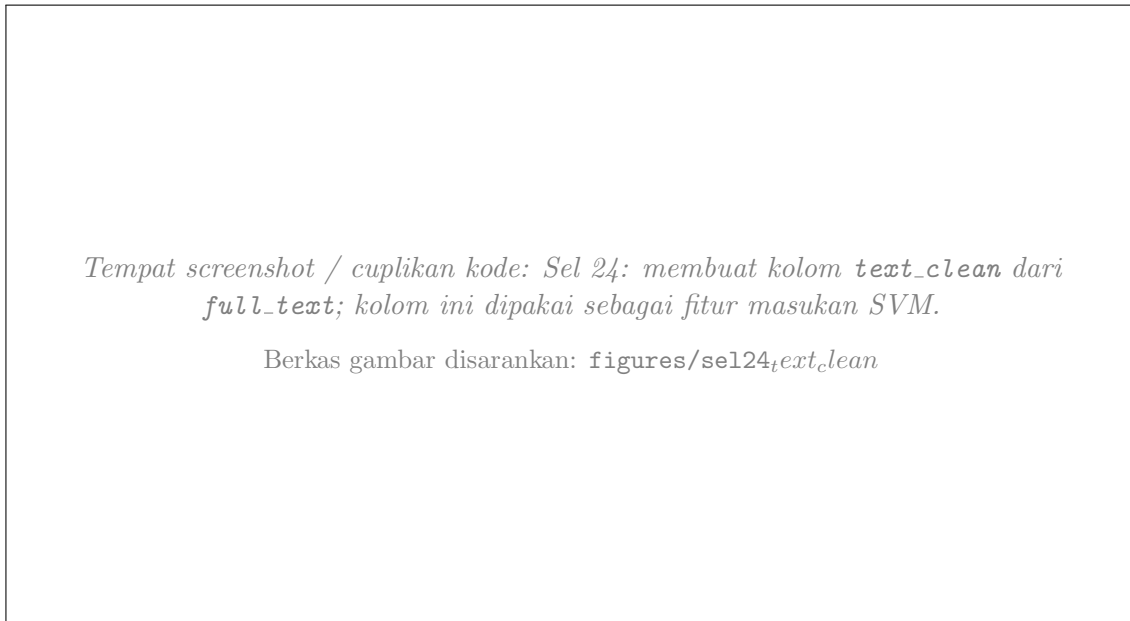


Figure 17: Sel 24: membuat kolom `text_clean` dari `full_text`; kolom ini dipakai sebagai fitur masukan SVM.

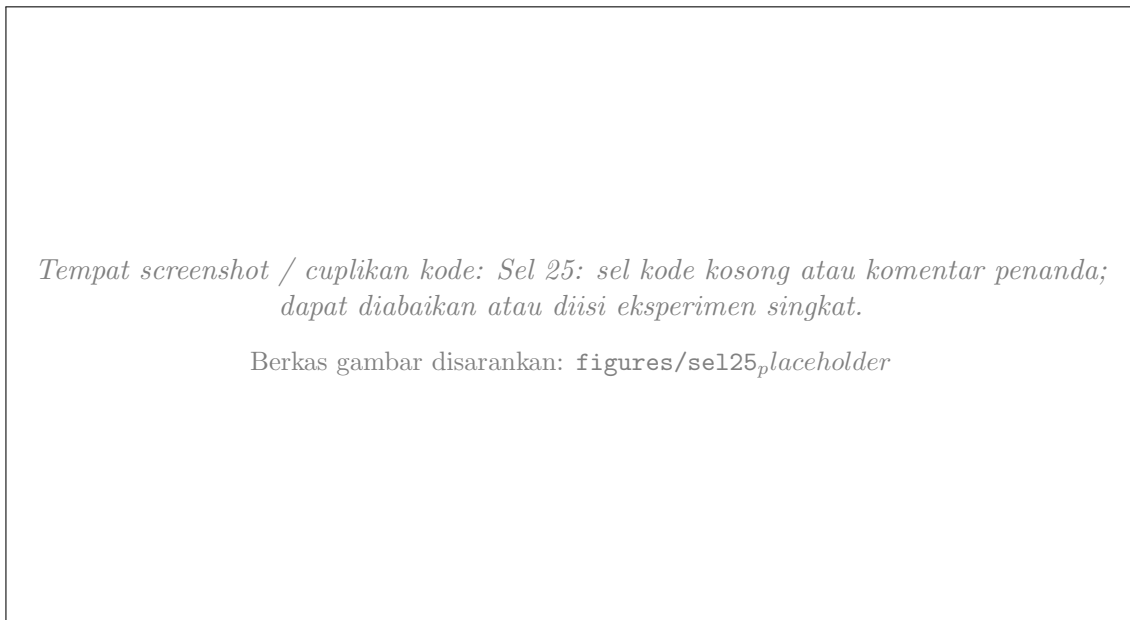


Figure 18: Sel 25: sel kode kosong atau komentar penanda; dapat diabaikan atau diisi eksperimen singkat.

*Tempat screenshot / cuplikan kode: Sel 27: integrasi `google.generativeai`;
normalisasi label; parsing JSON array respons LLM
(`parse_llm_sentiment_labels_json`, `extract_first_json_array`); pembagian batch;
penyimpanan respons mentah ke `raw_response`/; rekursi pemecahan batch jika parsing
gagal; penanganan kuota; pengisian kolom `label` pada `DataFrame`.*

Berkas gambar disarankan: `figures/sel27_autolabel`

Figure 19: Sel 27: integrasi `google.generativeai`; normalisasi label; parsing JSON array respons LLM (`parse_llm_sentiment_labels_json`, `extract_first_json_array`); pembagian batch; penyimpanan respons mentah ke `raw_response`/; rekursi pemecahan batch jika parsing gagal; penanganan kuota; pengisian kolom `label` pada `DataFrame`.

*Tempat screenshot / cuplikan kode: Sel 29: `df['label'].unique()` untuk memeriksa
kelas setelah autolabel (atau jika label sudah ada di file).*

Berkas gambar disarankan: `figures/sel29_label_unique`

Figure 20: Sel 29: `df['label'].unique()` untuk memeriksa kelas setelah autolabel (atau jika label sudah ada di file).

Tempat screenshot / cuplikan kode: Sel 30: pemetaan label ke integer; Pipeline TF-IDF (1-2 gram) + SVC linear berbobot kelas seimbang; GridSearchCV pada kombinasi `max_features` dan `C`; pencarian `test_size` terbaik pada hold-out terstratifikasi; menyimpan model dan data uji terbaik.

Berkas gambar disarankan: `figures/sel30_svm_grid`

Figure 21: Sel 30: pemetaan label ke integer; Pipeline TF-IDF (1-2 gram) + SVC linear berbobot kelas seimbang; GridSearchCV pada kombinasi `max_features` dan `C`; pencarian `test_size` terbaik pada hold-out terstratifikasi; menyimpan model dan data uji terbaik.

Tempat screenshot / cuplikan kode: Sel 32: prediksi pada `X_test`; matriks kebingungan dengan heatmap Seaborn; cetak `classification_report` (precision, recall, F1); simpan gambar dan teks laporan ke `EXPORT_DIR`.

Berkas gambar disarankan: `figures/sel32_confusion`

Figure 22: Sel 32: prediksi pada `X_test`; matriks kebingungan dengan heatmap Seaborn; cetak `classification_report` (precision, recall, F1); simpan gambar dan teks laporan ke `EXPORT_DIR`.

Tempat screenshot / cuplikan kode: Sel 34: menggabungkan `text_clean` menjadi satu string, membuat wordcloud, menampilkan dan menyimpan PNG.

Berkas gambar disarankan: `figures/sel34_wordcloud`

Figure 23: Sel 34: menggabungkan `text_clean` menjadi satu string, membuat *wordcloud*, menampilkan dan menyimpan PNG.

Tempat screenshot / cuplikan kode: Sel 36: menyimpan CSV berlabel, sampel prapemrosesan, `summary.json` (akurasi hold-out, parameter terbaik, ukuran data), mengompres isi folder ekspor menjadi satu berkas ZIP.

Berkas gambar disarankan: `figures/sel36_exportzip`

Figure 24: Sel 36: menyimpan CSV berlabel, sampel prapemrosesan, `summary.json` (akurasi hold-out, parameter terbaik, ukuran data), mengompres isi folder ekspor menjadi satu berkas ZIP.

Tempat screenshot / cuplikan kode: Sel 37: `files.download` Colab untuk mengunduh arsip hasil; di lingkungan lokal, langkah ini dapat diganti membuka folder ekspor secara manual.

Berkas gambar disarankan: `figures/sel37_colab_download`

Figure 25: Sel 37: `files.download` Colab untuk mengunduh arsip hasil; di lingkungan lokal, langkah ini dapat diganti membuka folder ekspor secara manual.

2 Hasil analisis sentimen

2.1 Ringkasan alur dan interpretasi umum

Setelah seluruh pipeline dijalankan, **hasil analisis sentimen** dapat dibahas dalam kerangka berikut.

1. **Distribusi label (autolabel LLM).** Kolom `label` merepresentasikan sentimen tiap dokumen: *positif*, *netral*, atau *negatif*. Sebaiknya dilaporkan jumlah dan proporsi per kelas (bisa dihitung dengan `df['label'].value_counts(normalize=True)`) untuk melihat apakah data *imbalance* dan apakah autolabel menghasilkan pola yang masuk akal untuk domain SpayLater/ShopeePay.
2. **Pemilihan pembagian data dan model.** Notebook membandingkan beberapa nilai `test_size` dan memilih kombinasi yang memberi **akurasi hold-out tertinggi** setelah `GridSearchCV` pada data latih. Pada `summary.json` (dan keluaran Sel 30) tercatat `best_test_size`, `holdout_accuracy`, `best_cv_accuracy`, dan `best_params` — nilai inilah yang harus Anda kutip sebagai **hasil utama** kinerja SVM pada skenario data Anda.
3. **Matriks kebingungan.** Heatmap membandingkan label aktual (baris) dan prediksi (kolom). Elemen diagonal besar berarti klasifikasi benar untuk kelas tersebut; kesalahan dominan di luar diagonal mengindikasikan pasangan kelas yang sering tertukar (misalnya *netral* vs *positif*), yang relevan untuk membahas kualitas teks atau ambiguitas opini pengguna.
4. **Laporan klasifikasi (precision, recall, F1).** Untuk tiap kelas, *precision* menjawab seberapa dapat dipercaya prediksi positif untuk kelas itu; *recall* seberapa banyak kasus aktual yang tertangkap; *F1* menyeimbangkan keduanya. Pada dataset tidak seimbang, akurasi global saja bisa menyesatkan; F1 per kelas dan rata-rata makro/terbobot lebih informatif.
5. **Wordcloud.** Visual ini menonjolkan kata-kata frekuensi tinggi pada `text_clean` setelah normalisasi. Kata yang dominan dapat mendukung narasi apakah wacana publik lebih sering menyentuh tema bunga, limit, keterlambatan, promo, dan sebagainya — diselaraskan dengan distribusi sentimen.

6. **Keterbatasan.** Label dari LLM bergantung pada kualitas prompt dan kuota API; SVM pada TF-IDF memodelkan asosiasi leksikal tanpa konteks mendalam seperti model besar. Hasil bersifat *indikatif* untuk penelitian/exploratory analysis dan sebaiknya divalidasi dengan sampel anotasi manusia bila digunakan untuk keputusan penting.

2.2 Tabel ringkasan (isi dari notebook Anda)

Isi tabel berikut dengan angka yang muncul di keluaran Sel 30–32 / berkas `summary.json` setelah Anda menjalankan notebook.

Table 1: Ringkasan metrik evaluasi (lengkapi setelah eksekusi notebook)

Ukuran	Nilai
Jumlah sampel (<code>n_samples</code>)	...
Proporsi kelas negatif / netral / positif	...
<code>test_size</code> terpilih	...
Akurasi hold-out	...
Akurasi CV terbaik (latih)	...
Parameter terbaik (<code>max_features</code> , <code>C</code>)	...

2.3 Paragraf contoh pembahasan (sesuaikan angka)

Model SVM dengan vektorisasi TF-IDF pada teks yang telah dinormalisasi mencapai akurasi hold-out sebesar [...] dengan pembagian uji [... %]. Dari matriks kebingungan, kelas yang paling sulit dipisahkan adalah [...]. Wordcloud menunjukkan kata-kata seperti [...] mendominasi korpus, selaras dengan proporsi sentimen [...]. Autolabel Gemini mempercepat pelabelan skala besar; namun sampel validasi manual disarankan pada kasus ambigu.

Referensi kode: seluruh implementasi terperinci berada di `spaylater_sentiment_analysis_gemini.ipynb`.